THỰC HÀNH 4: NGUYÊN LÝ THIẾT KẾ & DESIGN PATTERNS (SOLID)

🎯 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài thực hành này, bạn sẽ:

1. ✅ **Nhận biết** code vi phạm SOLID
2. ✅ **Refactor** code theo Single Responsibility Principle (SRP)
3. ✅ **Áp dụng** Open/Closed Principle (OCP)
4. ✅ **Hiểu** Dependency Inversion Principle (DIP)
5. ✅ **Implement** Singleton Pattern (thread-safe)
6. ✅ **Implement** Factory Pattern

⏱️ **Thời gian ước tính:** 6-8 giờ

📋 **Kiến thức cần có trước:**

- Đã hoàn thành Thực hành 3
- Đã đọc Chương 4: SOLID & Design Patterns
- Hiểu về Interface, Abstract Class

🧪 PHẦN 1: HƯỚNG DẪN CHI TIẾT (STEP-BY-STEP)

LAB 4.1: Giải Cứu "God Object" (SRP Refactoring)

Mục tiêu: Tách class vi phạm SRP thành nhiều class

Bước 1: Phân Tích Code Cũ (Vi Phạm SRP)

Code cũ (Rác):

```java
package com.bootcamp.lab4;

public class OrderManager {
    public void process(String item, double price) {
        // 1. Logic tính toán
        double total = price * 1.1;

        // 2. Logic Lưu trữ (DB) -> Lẫn lộn vào đây
        System.out.println("Connecting SQL...");
        System.out.println("Insert into Orders values...");

        // 3. Logic Gửi Mail -> Lẫn lộn tiếp
        System.out.println("SMTP Server connecting...");
        System.out.println("Email sent: Order success!");
```

```
    }
}
```

Vấn đề:

- Class này có **3 trách nhiệm**:
  1. Tính toán (Business Logic)
  2. Lưu database (Data Access)
  3. Gửi email (Notification)
- Vi phạm **Single Responsibility Principle**
- Khó test, khó bảo trì

Bước 2: Tách Database (Repository Pattern)

Tạo file: `OrderRepository.java`

```java
package com.bootcamp.lab4;

public class OrderRepository {
    public void save(String order) {
        System.out.println("💾 [Database] Đang lưu: " + order);
        // Code thực tế sẽ kết nối database và insert
    }

    public String findById(String id) {
        System.out.println("💾 [Database] Đang tìm: " + id);
        return "Order data";
    }
}
```

Giải thích:

- Chỉ chịu trách nhiệm về **Data Access**
- Dễ thay đổi database (MySQL → MongoDB) mà không ảnh hưởng code khác

Bước 3: Tách Notification

Tạo file: `EmailService.java`

```java
package com.bootcamp.lab4;

public class EmailService {
    public void send(String to, String subject, String message) {
        System.out.println("📧 [Email] Gửi đến: " + to);
        System.out.println("   Tiêu đề: " + subject);
        System.out.println("   Nội dung: " + message);
        // Code thực tế sẽ gửi email qua SMTP
```

```
    }
}
```

Giải thích:

- Chỉ chịu trách nhiệm về **Notification**
- Dễ thay đổi cách gửi (Email → SMS) mà không ảnh hưởng code khác

Bước 4: Class Chính (Chỉ Điều Phối)

Tạo file: `OrderService.java`

```java
package com.bootcamp.lab4;

public class OrderService {
    // Dependency Injection: Nhận dependencies từ bên ngoài
    private OrderRepository repository;
    private EmailService emailService;

    // Constructor Injection (Khuyến nghị)
    public OrderService(OrderRepository repo, EmailService email) {
        this.repository = repo;
        this.emailService = email;
    }

    // Chỉ tập trung vào Business Logic
    public void process(String item, double price) {
        // 1. Tính toán (Business Logic)
        double total = price * 1.1;  // Thêm 10% phí
        String orderInfo = item + " -> " + total + " VND";

        // 2. Nhờ Repository lưu
        repository.save(orderInfo);

        // 3. Nhờ EmailService gửi thông báo
        emailService.send("customer@example.com",
                          "Đơn hàng thành công",
                          "Cảm ơn bạn đã đặt hàng: " + orderInfo);

        System.out.println("✅ Đơn hàng đã được xử lý!");
    }
}
```

Giải thích:

1. **Dependency Injection:**
    - Không tự tạo dependencies (`new OrderRepository()`)
    - Nhận từ bên ngoài qua constructor
    - Dễ test (có thể mock dependencies)

2. **Single Responsibility:**
    - Chỉ chịu trách nhiệm về **Business Logic**
    - Không biết chi tiết database hay email hoạt động thế nào

Bước 5: Test Refactored Code

Tạo file: `OrderApp.java`

```
package com.bootcamp.lab4;

public class OrderApp {
    public static void main(String[] args) {
        // Tạo dependencies
        OrderRepository repo = new OrderRepository();
        EmailService email = new EmailService();

        // Inject vào Service
        OrderService service = new OrderService(repo, email);

        // Sử dụng
        service.process("Laptop Dell", 15000000);
    }
}
```

Lợi ích sau khi refactor:

- ✅ Mỗi class một trách nhiệm
- ✅ Dễ test từng phần riêng biệt
- ✅ Dễ thay đổi (đổi database không ảnh hưởng email)
- ✅ Tuân thủ SRP

LAB 4.2: Thanh Toán Đa Kênh (Factory Pattern)

Mục tiêu: Ẩn giấu logic khởi tạo object

Bước 1: Tạo Interface

Tạo file: `PaymentMethod.java`

```
package com.bootcamp.lab4;

public interface PaymentMethod {
    void pay(double amount);
}
```

Bước 2: Tạo Implementations

Tạo file: `Momo.java`

```
package com.bootcamp.lab4;

public class Momo implements PaymentMethod {
    @Override
    public void pay(double amount) {
        System.out.println("📱 [Momo] Đang thanh toán: " + amount + "
VND");
        System.out.println("   ✅ Thanh toán thành công qua Momo!");
    }
}
```

Tạo file: `ZaloPay.java`

```
package com.bootcamp.lab4;

public class ZaloPay implements PaymentMethod {
    @Override
    public void pay(double amount) {
        System.out.println("💬 [ZaloPay] Đang thanh toán: " + amount + "
VND");
        System.out.println("   ✅ Thanh toán thành công qua ZaloPay!");
    }
}
```

Tạo file: `Cash.java`

```
package com.bootcamp.lab4;

public class Cash implements PaymentMethod {
    @Override
    public void pay(double amount) {
        System.out.println("💵 [Tiền mặt] Đang thanh toán: " + amount + "
VND");
        System.out.println("   ✅ Đã nhận tiền mặt!");
    }
}
```

Bước 3: Tạo Factory

Tạo file: `PaymentFactory.java`

```
package com.bootcamp.lab4;

public class PaymentFactory {
```

```
    // Static Factory Method
    public static PaymentMethod createPaymentMethod(String type) {
        // Logic tạo object được tập trung ở đây
        switch (type.toLowerCase()) {
            case "momo":
                return new Momo();
            case "zalo":
            case "zalopay":
                return new ZaloPay();
            case "cash":
            case "tienmat":
                return new Cash();
            default:
                throw new IllegalArgumentException("❌ Chưa hỗ trợ phương
thức: " + type);
        }
    }
}
```

Giải thích:

- Factory tập trung logic `switch-case` bẩn
- Client không cần biết cách tạo object
- Dễ thêm phương thức mới (chỉ sửa Factory)

Bước 4: Sử Dụng Factory

Tạo file: `ShopApp.java`

```
package com.bootcamp.lab4;

import java.util.Scanner;

public class ShopApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== CỬA HÀNG TRỰC TUYẾN ===\n");
        System.out.print("Nhập số tiền: ");
        double amount = sc.nextDouble();

        System.out.print("Chọn phương thức thanh toán (momo/zalo/cash):
");
        String paymentType = sc.next();

        // Factory tạo PaymentMethod
        PaymentMethod payment =
PaymentFactory.createPaymentMethod(paymentType);

        // Sử dụng - Không cần biết là Momo hay ZaloPay
        payment.pay(amount);
```

```
        sc.close();
    }
}
```

Lợi ích:

- ✅ Client code sạch (không có if/else)
- ✅ Dễ mở rộng (thêm phương thức mới)
- ✅ Tuân thủ Open/Closed Principle

LAB 4.3: Singleton Pattern (Logger)

Mục tiêu: Đảm bảo chỉ có 1 instance duy nhất

Bước 1: Singleton Eager Initialization

Tạo file: `Logger.java`

```java
package com.bootcamp.lab4;

public class Logger {
    // Eager Initialization: Tạo instance ngay khi class được load
    private static final Logger instance = new Logger();

    // Private constructor - Không cho tạo từ bên ngoài
    private Logger() {
        System.out.println("Logger đã được khởi tạo");
    }

    // Public method để lấy instance
    public static Logger getInstance() {
        return instance;
    }

    // Business methods
    public void log(String message) {
        System.out.println("[LOG] " + message);
    }

    public void error(String message) {
        System.out.println("[ERROR] " + message);
    }

    public void info(String message) {
        System.out.println("[INFO] " + message);
    }
}
```

Giải thích:

- `private constructor`: Không thể `new Logger()` từ bên ngoài
- `static final instance`: Chỉ có 1 bản copy duy nhất
- `getInstance()`: Cách duy nhất để lấy instance

Bước 2: Test Singleton

Tạo file: `LoggerTest.java`

```
package com.bootcamp.lab4;

public class LoggerTest {
    public static void main(String[] args) {
        // Lấy instance
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        // Kiểm tra cùng một instance
        System.out.println("logger1 == logger2: " + (logger1 == logger2));
// true

        // Sử dụng
        logger1.log("Message 1");
        logger2.log("Message 2");
        logger1.info("Thông tin quan trọng");
        logger2.error("Có lỗi xảy ra!");
    }
}
```

Kết quả:

```
Logger đã được khởi tạo
logger1 == logger2: true
[LOG] Message 1
[LOG] Message 2
[INFO] Thông tin quan trọng
[ERROR] Có lỗi xảy ra!
```

Lưu ý: Logger chỉ được khởi tạo 1 lần duy nhất!

📝 PHẦN 2: BÀI TẬP CÓ HƯỚNG DẪN

Bài 4.1: Refactor theo SRP

Đề bài: Refactor class `User` sau:

```java
public class User {
    private String name;
    private String email;

    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }

    public void saveToDatabase() {
        // Code lưu database
    }

    public void sendEmail(String message) {
        // Code gửi email
    }

    public void generateReport() {
        // Code tạo report
    }
}
```

Hướng Dẫn

Tách thành:

1. `User` - Chỉ quản lý thông tin
2. `UserRepository` - Lưu database
3. `EmailService` - Gửi email
4. `ReportGenerator` - Tạo report

Code sau khi refactor:

```java
// User.java - Chỉ quản lý thông tin
package com.bootcamp.lab4;

public class User {
    private String name;
    private String email;

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
}

// UserRepository.java - Lưu database
```

```java
package com.bootcamp.lab4;

public class UserRepository {
    public void save(User user) {
        System.out.println("💾 Đang lưu user vào database: " +
user.getName());
    }

    public User findById(String id) {
        System.out.println("💾 Đang tìm user: " + id);
        return new User("Found", "found@example.com");
    }
}

// EmailService.java - Gửi email
package com.bootcamp.lab4;

public class EmailService {
    public void sendEmail(String to, String message) {
        System.out.println("📧 Gửi email đến: " + to);
        System.out.println("   Nội dung: " + message);
    }
}

// ReportGenerator.java - Tạo report
package com.bootcamp.lab4;

public class ReportGenerator {
    public void generateReport(User user) {
        System.out.println("📄 Tạo report cho: " + user.getName());
    }
}
```

Bài 4.2: Open/Closed Principle

Đề bài: Refactor `AreaCalculator` để tuân thủ OCP

Code cũ:

```java
public class AreaCalculator {
    public double calculateArea(Object shape) {
        if (shape instanceof Circle) {
            Circle c = (Circle) shape;
            return Math.PI * c.getRadius() * c.getRadius();
        } else if (shape instanceof Rectangle) {
            Rectangle r = (Rectangle) shape;
            return r.getWidth() * r.getHeight();
        }
        return 0;
    }
}
```

Hướng Dẫn

Bước 1: Tạo Interface/Abstract Class

```
package com.bootcamp.lab4;

public abstract class Shape {
    public abstract double calculateArea();
}
```

Bước 2: Implement ở Subclass

```
package com.bootcamp.lab4;

public class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    public double getRadius() { return radius; }
}

// Rectangle tương tự...
```

Bước 3: Calculator Đơn Giản

```
package com.bootcamp.lab4;

public class AreaCalculator {
    // Không cần sửa khi thêm loại mới!
    public double calculateArea(Shape shape) {
        return shape.calculateArea();  // Polymorphism!
    }
}
```

Lợi ích:

- ✅ Thêm `Triangle` → Chỉ cần tạo class mới, không sửa `AreaCalculator`

- ✅ Tuân thủ Open/Closed Principle

🎯 PHẦN 3: BÀI TẬP TỰ LUYỆN (KHÔNG CÓ HƯỚNG DẪN)

Bài 4.1: Refactor God Object

Đề bài: Refactor class `ProductManager` vi phạm SRP:

```
public class ProductManager {
    private String name;
    private double price;

    public void setName(String name) { this.name = name; }
    public void setPrice(double price) { this.price = price; }

    public void saveToFile() { /* ... */ }
    public void sendToWarehouse() { /* ... */ }
    public void calculateTax() { /* ... */ }
    public void generateInvoice() { /* ... */ }
}
```

Yêu cầu:

- Tách thành nhiều class
- Mỗi class một trách nhiệm

Bài 4.2: Factory Pattern cho Database

Đề bài: Tạo Factory cho Database connections:

1. **Interface `Database`:**
   - Methods: `connect()`, `query(String sql)`, `close()`
2. **Implementations:**
   - `MySQLDatabase`
   - `PostgreSQLDatabase`
3. **Factory `DatabaseFactory`:**
   - Method: `createDatabase(String type)`

Yêu cầu:

- Sử dụng Factory Pattern
- Dễ thêm database mới

Bài 4.3: Singleton Configuration

Đề bài: Tạo class `Config` dùng Singleton:

- Attributes: `databaseUrl`, `apiKey`, `timeout`
- Methods: `getDatabaseUrl()`, `setDatabaseUrl()`, ...
- Đảm bảo chỉ có 1 instance

Yêu cầu:

- Thread-safe Singleton
- Có thể load config từ file

Bài 4.4: Refactor theo OCP (Open/Closed Principle)

Đề bài: Có class `AreaCalculator` tính diện tích:

```
public double calculateArea(String shape, double... params) {
    if (shape.equals("circle")) {
        return Math.PI * params[0] * params[0];
    } else if (shape.equals("rectangle")) {
        return params[0] * params[1];
    }
    // Phải sửa code khi thêm hình mới
}
```

Yêu cầu:

- Refactor để tuân thủ OCP
- Thêm hình mới (Triangle) mà không sửa code cũ
- Dùng Abstract class hoặc Interface

Gợi ý:

- Tạo interface `Shape` với method `getArea()`
- Mỗi hình implement interface này
- `AreaCalculator` chỉ nhận `Shape` parameter

Bài 4.5: Dependency Injection cho Email Service

Đề bài: Refactor class `UserService` phụ thuộc trực tiếp vào `EmailService`:

```
public class UserService {
    private EmailService emailService = new EmailService(); // Hard
dependency

    public void registerUser(String email) {
        // ... logic
        emailService.sendEmail(email);
```

```
    }
}
```

Yêu cầu:

- Tách dependency bằng Dependency Injection
- Tạo interface `EmailSender`
- `EmailService` implement `EmailSender`
- Inject qua constructor hoặc setter

Gợi ý:

- Constructor injection: `UserService(EmailSender emailSender)`
- Hoặc setter injection: `setEmailSender(EmailSender emailSender)`

Bài 4.6: Factory Pattern cho Logger

Đề bài: Tạo hệ thống logging với Factory:

1. **Interface `Logger`:**
    - Methods: `info(String msg)`, `error(String msg)`, `debug(String msg)`
2. **Implementations:**
    - `ConsoleLogger`: In ra console
    - `FileLogger`: Ghi vào file
    - `DatabaseLogger`: Lưu vào database
3. **Factory `LoggerFactory`:**
    - Method: `createLogger(String type)`
    - Method: `createLoggerFromConfig()`: Đọc từ config file

Gợi ý:

- Factory ẩn logic tạo object
- Dễ thêm loại logger mới
- Có thể dùng Singleton cho FileLogger và DatabaseLogger

Bài 4.7: SRP - Tách Validation Logic

Đề bài: Class `User` vi phạm SRP:

```
public class User {
    private String email;
    private String password;

    public void setEmail(String email) {
```

```
        // Validation logic ở đây
        if (!email.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        // ... nhiều validation khác
        this.email = email;
    }

    public void save() { /* Save to database */ }
    public void sendWelcomeEmail() { /* Send email */ }
}
```

Yêu cầu:

- Tách validation logic ra class riêng
- Tách persistence logic ra class riêng
- Tách email logic ra class riêng
- `User` chỉ giữ data và business logic cơ bản

Gợi ý:

- `UserValidator`: Chứa validation methods
- `UserRepository`: Chứa save/load methods
- `EmailService`: Chứa email methods

Bài 4.8: Builder Pattern cho Complex Object

Đề bài: Tạo class `Computer` với nhiều thuộc tính:

- `cpu`, `ram`, `storage`, `gpu`, `motherboard`, `powerSupply`
- Một số thuộc tính bắt buộc, một số optional
- Constructor với nhiều tham số khó đọc

Yêu cầu:

- Implement Builder Pattern
- Method chaining: `new Computer.Builder().cpu("Intel").ram(16).build()`
- Validation: CPU và RAM là bắt buộc

Gợi ý:

- Inner class `Builder` trong `Computer`
- Method `build()` trả về `Computer` instance
- Mỗi setter trả về `this` để chain

🏆 PHẦN 4: THỬ THÁCH NÂNG CAO

Challenge 1: Smart Home (Command Pattern)

Đề bài: Thiết kế hệ thống điều khiển nhà thông minh.

Yêu cầu:

1. **Interface `Command`:**
   - Method: `execute()`
2. **Commands:**
   - `TurnOnLightCommand`
   - `TurnOffLightCommand`
   - `OpenDoorCommand`
3. **Class `RemoteControl`:**
   - Method: `setCommand(Command cmd)`
   - Method: `pressButton()`: Thực thi command

Mục tiêu: Remote không biết Light hay Door, chỉ biết gọi `execute()`

Challenge 2: Strategy Pattern

Đề bài: Tạo hệ thống gửi thông báo với Strategy Pattern.

Yêu cầu:

1. **Interface `NotificationStrategy`:**
   - Method: `send(String message)`
2. **Strategies:**
   - `SMSStrategy`
   - `EmailStrategy`
   - `PushNotificationStrategy`
3. **Class `User`:**
   - Có thể thay đổi strategy tại runtime
   - Method: `setNotificationStrategy(NotificationStrategy strategy)`
   - Method: `notify(String message)`

Mục tiêu: User có thể đổi cách nhận thông báo mà không cần thay đổi code

---

📚 BẢNG GHI NHỚ (CHEAT SHEET)

SOLID Principles

Nguyên lý	Ý nghĩa	Cách nhận biết vi phạm
S - SRP	Một class một trách nhiệm	Class có nhiều lý do để thay đổi
O - OCP	Mở cho extension, đóng cho modification	Phải sửa code cũ khi thêm tính năng mới

Nguyên lý	Ý nghĩa	Cách nhận biết vi phạm
D - DIP	Phụ thuộc vào abstraction	High-level module phụ thuộc low-level

Design Patterns

Pattern	Mục đích	Khi nào dùng
Singleton	Đảm bảo 1 instance	Logger, Config, Database Connection
Factory	Ẩn logic tạo object	Không biết trước loại object cần tạo
Strategy	Thay đổi algorithm tại runtime	Nhiều cách thực hiện cùng một việc

---

✅ CHECKLIST HOÀN THÀNH

- ☐ Đã hiểu SRP và refactor code vi phạm
- ☐ Đã áp dụng OCP với Polymorphism
- ☐ Đã hiểu DIP và Dependency Injection
- ☐ Đã implement Singleton Pattern
- ☐ Đã implement Factory Pattern
- ☐ Đã hoàn thành LAB 4.1 (OrderService)
- ☐ Đã hoàn thành LAB 4.2 (PaymentFactory)
- ☐ Đã hoàn thành LAB 4.3 (Logger Singleton)
- ☐ Đã làm ít nhất 2 bài tập có hướng dẫn
- ☐ Đã làm ít nhất 2 bài tập tự luyện
- ☐ Đã thử ít nhất 1 thử thách nâng cao

🎓 TỔNG KẾT

Sau bài thực hành này, bạn đã:

1. ✅ **Nhận biết** code vi phạm SOLID
2. ✅ **Refactor** code theo SRP
3. ✅ **Áp dụng** OCP và DIP
4. ✅ **Implement** Singleton và Factory Pattern
5. ✅ **Tư duy thiết kế** chuyên nghiệp

Bước tiếp theo: Chuyển sang Thực hành 5: Xử lý lỗi & Unit Testing

---

Chúc bạn học tốt! 🚀